8/1/2025

Lecture 1

Topics that will be covered

Various design techniques :-

1. Divide and Conquer → Merge Sort
2. Greedy Method
3. Dynamic Programming → Storing some imp. data
4. Network Flow Based Optimization
   ↳ Red^n to network flow problem from other problems (Matching → LPnC Problem).

Above are classical Algo Design Paradigm

5. Intractability

Eg:- $(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee \neg x_4)$

$x_1 = T$
$x_4 = F$

m = # of clauses
n = # of variables

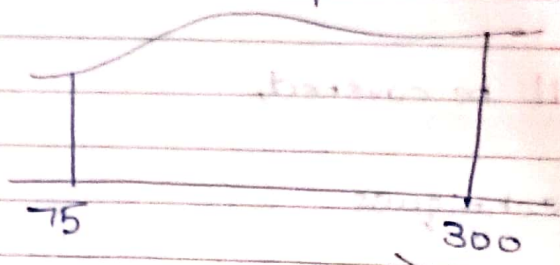
Sol^n for Satisfiability (Assignment for expres^n to be true)

Algorithmic → Exponential (2^n) Time/Computations

6. Randomisation → Helps in speeding up computations

Eg: $y = \tan x \times x^2 + \frac{x^3 - 3x^2}{\cos x}$

Find area b/w 75 to 300



7 Approximation — Randomisation

Randomly generate (x,y)

$\frac{\text{Unknown area}}{\text{Known area}}$ = Prob. (x,y) in given area

Generate 50,100 random number and find the probability.

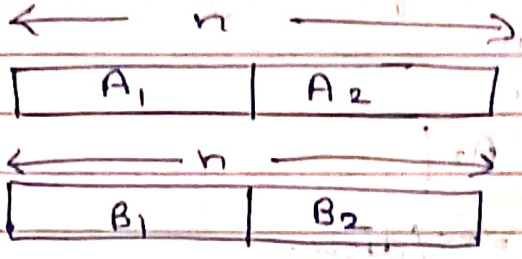
* Divide and Conquer

1 Multiplication of decimals

Long multiplication takes $O(n^2)$

n = # of digits in the 2 nos.

Multipl^n can be done as fast as add^n



$$P = 10^n(A_1B_1) + 10^{n/2}(A_1B_2 + A_2B_1) + A_2B_2$$

$$56 \times 35$$

$$P = (5\times3)\times100 + (5\times5 + 6\times3)\times10 + 6\times5$$

* Does this breaking speed up the computation process?

9/1/2024

Lecture 2

* Correctness Proof of Algo.
  ↳ PMI

35 % Mid Sem
35 % End Sem
30 % - Others (Class Participation + Quizzes)

* Divide and Conquer Algorithm for integer multiplication

$A = [A_1 : A_2]$ — n bits/digits

$B = [B_1 : B_2]$ — n bits/digits

$A \cdot B = (A_1 \times 10^{n/2} + A_2)(B_1 \times 10^{n/2} + B_2)$

$= A_1B_1 \times 10^n + (A_2B_1 + A_1B_2) \times 10^{n/2} + A_2B_2$

4 multiplications
3 additions
Same amount of time because each bit/digit gets multiplied with every digit of other no.

$T(n)$ → Worst case time

$T(n) \leq 4T(n/2) + 20 \cdot n$

For multiplying 2 n digit no.s → Addit^n and Shifting time

T

Claim :- $T(n) \leq 5n^2$

$$T(n) \leq 4\left(4T\left(\frac{n}{4}\right) + 20\frac{n}{2}\right) + 20n$$

$$\leq 16\,T\left(\frac{n}{4}\right) + 40n + 20n$$

$$16 \times 20 \times \frac{n}{4} + 40n + 20n$$

$$\leq 64\,T\left(\frac{n}{8}\right) + 80n + 40n + 20n$$

$$\leq a \cdot T\left(\frac{n}{b}\right) + \cdots + 160n + 80n + 40n + 20n$$

$$\left(\log_b a\right) n^2$$

$$\therefore T\left(\frac{n}{2}\right) \leq c\left(\frac{n}{2}\right)^2 = \frac{cn^2}{4}$$

$$4\left(T\left(\frac{n}{2}\right)\right) + 20n \leq \frac{4cn^2}{4} + 20n$$

$$\leq cn^2 + 20n$$

* Way to reduce multiplication to 3 $T\left(\frac{n}{2}\right)$

$$(A_1 + A_2)(B_1 + B_2) = A_1B_1 + A_1B_2 + A_2B_1 + A_2B_2$$

1 2

3rd multiplication

$$A_1B_2 + A_2B_1 = (A_1 + A_2)(B_1 + B_2) - A_1B_1 - A_2B_2$$

$$\Rightarrow T(n) \leq 3\,T\left(\frac{n}{2}\right) + 20n$$

10/1/2024

Lecture 3

Recap

Fast Integer Multiplⁿ

Idea

Split the integers A & B into equal halves say A_1, A_2 & B_1, B_2

We need the following terms

* A_1B_1 , A_2B_2 , $A_1B_2 + A_2B_1$

These can be computed using 3 multiplⁿ instead of 4.

Multiply (A, B) → Constant:

Base Case ← if A & B are less than 5 digits, return A·B

O(n) { $A_1, A_2 \leftarrow$ (first half of A, second ½ of A)

$B_1, B_2 \leftarrow$ (" " " B, " " " B)

$c_1 \leftarrow$ Multiply (A_1, B_1)

$c_2 \leftarrow$ " (A_2, B_2)

$c_3 \leftarrow$ " $((A_1+A_2), (B_1+B_2))$ } $3T(n/2)$

Compute (c_1, c_2, c_3) $O(n)$

$T(n) \leq 3T\left(\frac{n}{2}\right) + O(n)$

$T(n) \leq n^{\log_2 3}$ Solved using Master's Theorem

$T(n) \leq n^{1.59}$

Split into 3 parts but we need to achieve it in 5 computations

Asymptotically $n^{0.001} > (\log n)^{500}$

* Randomised Quick Sort

Expected Worst Case Time Complexity of Rand. Quick Sort = $O(n \log n)$

* Problem 1

Check if n is of the form 3^{43^k}

Algo check (n)
print ("No")

Problem → Also have 1 good coin

* 13 coins out of which 1 is counterfeit. Identify it. Beam Balance is ~~counterf~~ available. Minimise the no. of weighing.

1 weighing gives → 3 info (possible)

$\frac{3^n - 1}{2}$ coins : 1 counterfeit

E.g. 40 coins 13 13 14

↑ 13+G 14 ↓ Case 1

13+G = 14 Case 2

↓ 13+G 14 ↑ Case 3

Find faulty version 1 (13 coins) Case 2

" " version 2 (~~(13 PLight)~~ 14, ~~possibly~~ heavy) Case 1

version 3 ((13 PH), (14 PL)) Case 3

Prob 1
27 coins
1 counterfeit
→ Harder bcoz do not know whether faulty

Prob 2 — Easier version

↑ 14 → 13 + G, ↓

↑ A A₂
2

A₁ = A₂
→ faulty

1 + G, ↓

↑ A₁ A₂ ↓
↓ faulty.

16/1/2024

Lecture 4

Date __/__/__
Page _____

* Problem Description
We are given N coins. All but one coin is good ~~fone~~ and has weight different from the others. Identify the counterfeit coin using beam balance.

Obj: Minimise the no of uses of the balance.

E.g. 3 coins with heavier coin
↳ 1 use
4 coins with heavier coin
↳ 1 use is not perfect bcoz ∄ 1 use gives 3 possibilities
4 to 9 → 2 uses

Variant I:- We know "type" of error
II:- we do not know "type" of error

* Search Space → (I) → All the coins can be heavy
↘
N

II {1, H} {1, L} → 2N
{N, H} {N, L}

2 Coins counter feit
H/L H/L

$\binom{N}{2} \times 4 = 2 \times 2 = 4 \frac{N(N-1)}{2} = 2N(N-1)$

maximind

If $N = \frac{3^n - 1}{2}$

:

Variant I N coins → Uses $= \lceil \log_3 N \rceil$

Record the uses of balance part Left, goes Heavy, Light, Equal

1st use
2nd
:
Kth

} 3, 3, .. 3^K possibilities

$3^K \geq N$

If $3^K < N$ then some balls will not be able to appear in table so lower bounded by $\lceil \log_3 N \rceil$

* $\lceil \frac{N}{3} \rceil$, $\lceil \frac{N}{3} \rceil$, $N - 2\lceil \frac{N}{3} \rceil$ batches of coins

~~This~~ After 1 weighing we recognise the faulty batch

Again divide the batch in 3 batches

⇓

Recursively $\lceil \log_3 N \rceil$ uses.

Variant 2 38 coins

No. of possibilities = 76

13 13 12

i | Light → H | Heavy → L

1 – 13 | 14 – 26

13 poss + 13 poss = 26 poss

↓ Reduce to 9 poss

1 – 4 | 14 – 18 | 5 – 8 + 19 – 23 | Good Coin

4H 5L | 4H 5L | If =

↓ | 10, 13 H → 8 poss

1–4 H or 19–22 L | 24 – 26 L 8 poss

↓

9 poss

5 – 9 + 14 – 18 = 9 poss

⇒ 5H 4L If equal → 9 → 3

2H 1L | 2H 1L ↑

↓

2H 1L

Compare these 2

Similarly $\lceil \log_3 2N \rceil$

$\frac{3^{n+1}-1}{2}$ no. of coins

Search space size $= 3^{n+1}-1$ | Good coin

$\frac{3^n-1}{2}$ $\frac{3^n-1}{2}$ $\frac{3^n+1}{2}$

A B C

$\frac{3^{n-1}-1+3^n+1}{2} = 3^n$ Size when beams balance

$= 3^n-1$ when beams balance

Similarly recursion & just reduce the size of problem.

$\alpha = \frac{3^n-1}{2}$ ... ⊗ $\beta = \frac{3^n+1}{2}$

H L

H_1 H_2 H_3 L_1 L_2 L_3

⊗ H_1 L_1 H_2 L_2

similar comparison follows as in case of 38 coins.

Maintain difference of 1 coin among groups as in 38 coin example.

Self Notes

$\log_3 \lceil 76 \rceil = 4$

38 balls → 76 possibilities

Case I :

i) 13 = 13 12

↳ 12 → 24 poss

ii) 4 = 4 4

↳ (4) → 2 weighings

iii) 8 pos

2 1 = 1 2

1 1 → This weighing gives us the answer

Case II 13 > 13 12

(i) 13H 13L → 26 poss

9 9 8

(ii) 5H 4L = 5H 4L → 3H 5L

(iii) 1H2L = 1H 2L → 1H 1L

↓

(iv) Weight any with known one

(iii) 1H 2L > 1H 2L → 1H 1L

↓

1H 2L → Compare these 2 = then heavy

(iv) umbalance even light is faulty

(ii) 5H 4L > 5H 4L

↓

5H 4L → 2H 1L 2H 1L 1H 2L

Equal, >, < leaves us with 3 poss which can be resolved

Lecture 5 17/1/2024

Date / / Page

* $\frac{3^n - 1}{2}$ $n = 93$ ~~40~~ 13 coins

$\lceil \log_3 80 \rceil = 4$ weights

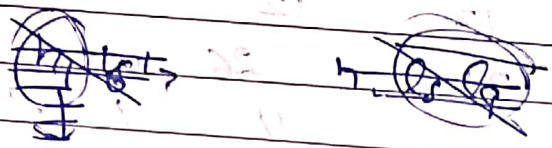
~~13~~ ~~13~~ ~~14~~

4 4 5 → 26

∴ +1 Good Coin

$h_1 h_2 h_3 h_4$ $l_6 l_7 l_8 l_9 l_{10}$

$h_1 l_6 l_7$, $h_2 l_8 l_9$, $h_3 h_4 l_{10}$



13 13 14 → 80

40 40 41 → 242

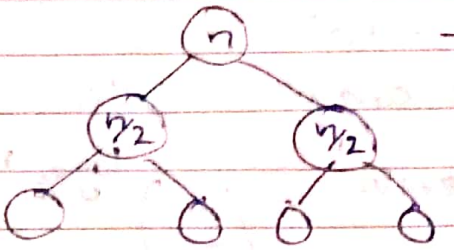
* The algo ~~takes~~ divides the pop from 242 to 80 to ~~27~~ 26. ⇒ $\log_3 \lceil 2N \rceil$

* Quick-Sort

$T(n) \leq 2T\left(\frac{n}{2}\right) + cn$

~~Ethy~~ Quick-sort $O(n) = n \log n$ if we are able to find any Kth ranked element in the array every time.

maximind



— cn cost

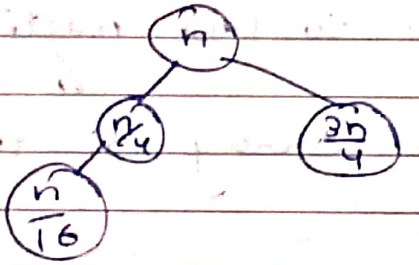
— $2 \frac{cn}{2}$ cost

cn

c · n · (no. of levels) = O(n log n)

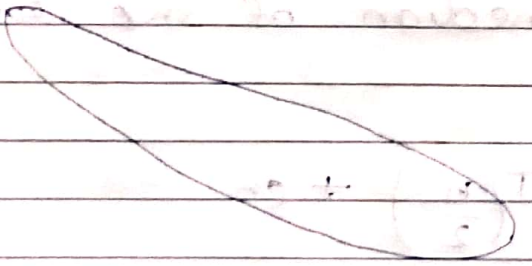
If symmetric division is not there

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + kn$$



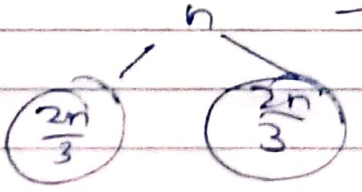
kn

kn



O(n) = kn · log n

$$T(n) = 2\,T\left(\frac{2n}{3}\right) + cn$$



— cn

— $\frac{4cn}{3}$

— $\frac{16\,cn}{9}$

$\Rightarrow 1 + a + a^2 + \ldots\ldots + a^n = \frac{}{} a^n$

Asymptotic analysis $\longrightarrow$

$\left(\frac{4}{3}\right)^{\text{no. of levels}} = \left(\frac{4}{3}\right)^{\log_{3/2} n}$

$= n^{\log_{3/2}(4/3)}$

Finding Kth ~~elt~~ ranked elt.

* Problem Statement

Input: Array A with n elts, $1 \le k \le n$

Output: k^{th} largest elt of A.

We have algo to find median

Then partition array on the basis of median

$O(n)$

Then find median of the respective array.

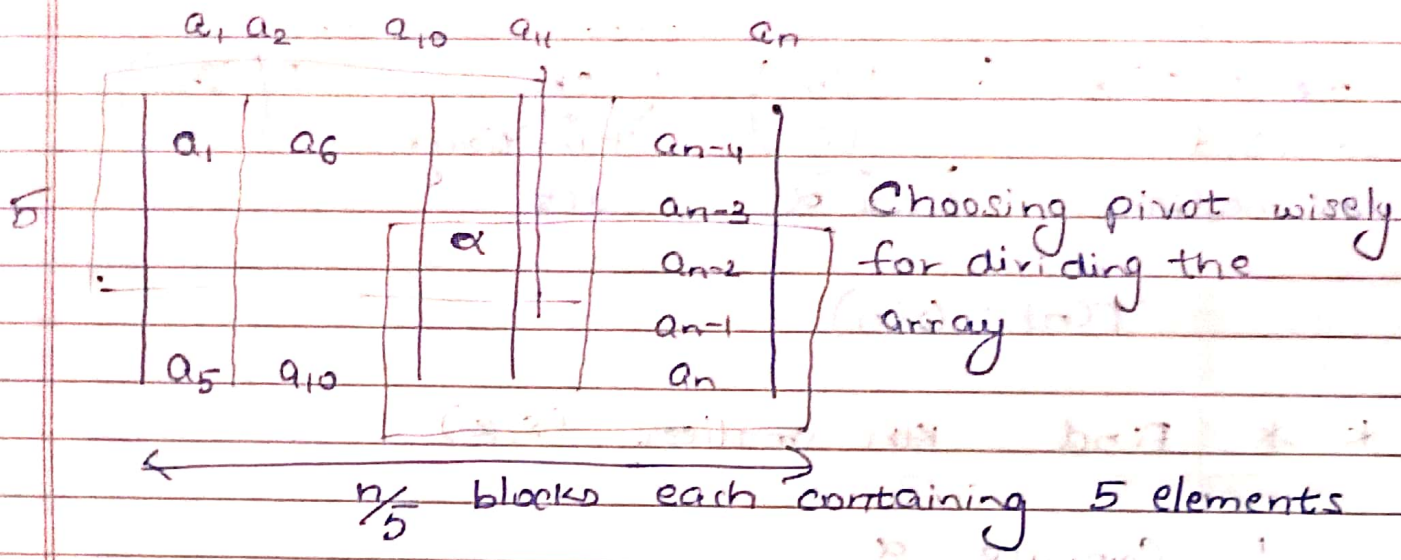
$$T(n) \le T\left(\frac{n}{2}\right) + cn$$

To find median $\longrightarrow$ Blum's Selection Algorithm.

Lecture 6 18/1/2024

Continued

Blum's Selection Algorithm



Medians of each blocks are :-

$b_1 \; b_2 \; \cdots \; b_{n/5}$ → c_n Finding the medians $b_1 \; b_2 \; \cdots \; b_{n/5}$

Subproblem → Finding median of b_i → $T(n/5)$

Let α be the median of $b_1 \cdots b_{n/5}$

Use α as the pivot and position the array on its basis

$$\max\{|S_<|, |S=|, |S>|\}$$

Guaranteed than $(|S_<| \cdot |S_>|) \rightarrow \frac{3}{5} \times \frac{n}{10}$

$$= \frac{3n}{10}$$

$$|S_>| \leq n - \frac{3n}{10} = \frac{7n}{10}$$

$\frac{3n}{10} \leq |S_1|, |S_2| \leq \frac{7n}{10}$

Time to find the median $\leftarrow T(n) \leq T\left(\frac{n}{5}\right)$ + cn + $T\left(\frac{7n}{10}\right)$

$T\left(\frac{n}{5}\right)$: Find α; cn: Partitioning cost

$T(n) = O(n)$

** Find Kth smallest (S, K)

1 Compute α

1.1 Split S into $\frac{n}{5}$ blocks

$T\left(\frac{n}{5}\right)$ 1.2 Compute median of each block say S_1

1.3 $\alpha \leftarrow$ Find Kth smallest $\left(S_1, \frac{n}{10}\right)$

2 Partition on the basis of α

$O(n)$ Scan through S to generate S_1 S_2 $S_=$

3 Based on the sizes of $S_1, S_2, S_=$ determine the component in which we

$O(1)$ need to search

$P_3[K_1] = f(|S_1|, |S_2|, |S_=|, K)$

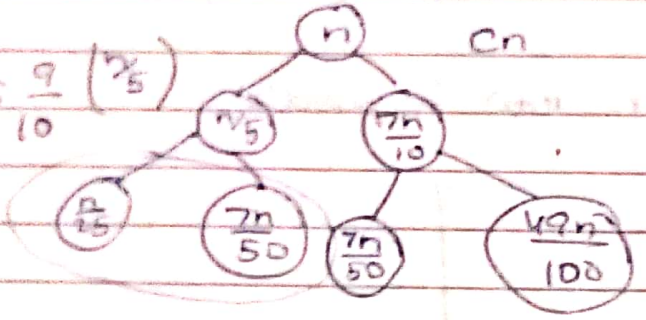
4 Recursion Find - Kth - smallest (S_p, K_1)

$T\left(\frac{7n}{10}\right)$

$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + cn$

Each time problem size is ting by 90%

⇒ Linear time



$c\frac{9}{10}\left(\frac{n}{5}\right)$

$\frac{cn}{5} + \frac{c7n}{10} = \frac{c9n}{10}$

49
+ 28
+ 4
81

$c\frac{81n}{100}$

$cn\left[1 + \frac{9}{10} + \frac{81}{100} \cdots\right]$

$c\frac{9}{1-}$

$\frac{a^{n+1} - 1}{a - 1} = 1 -$

$a -$

O(n) show

Total cost = $(10c+1)n$
= $O(n)$

** Fibonacci Series

Lecture 7

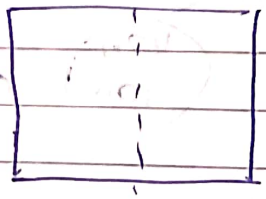
23/1/2024

Date / /
Page

* Closest Pair of Points

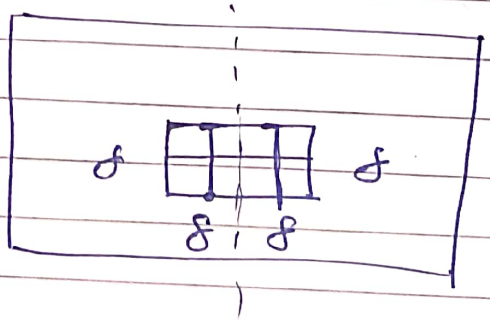
Input :- A set of n points in 2-D.
Output :- A pair of points which are closest. (Euclidean Distance)

Divide region into 2 parts.



δ_L = mindistance in left half
δ_R = " " " Right "

$\delta = \min(\delta_L, \delta_R)$



8 squares of size $\frac{\delta}{2}$

Each square can contain at max 1 point.

Closest Pair (S_x, S_y)

$P_1^L P_2^L$ δ_L ← Closest Pair (S_x^L, S_y^L)

$P_1^R P_2^R$ δ_R ← Closest Pair (S_x^R, S_y^R)

2 Compute the 2δ band sorted by y coordinate

3 For every point in the band ~~updat~~ compute distance @ in the next δ elts. in the band and update δ.

Lecture 8

24/1/2024

Input:- A collection S of points in 2D which is sorted by x coordinates & y coordinates

output :- (P_1, P_2, δ) s.t. $d(P_1, P_2) = \delta$ & for all $\alpha, \beta \in S$ $d(\alpha, \beta) \geq \delta$

Closest Pair (S_x, S_y)

0 Take care of base cases

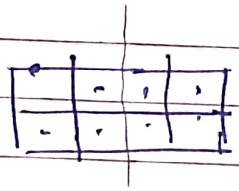
$T(\frac{n}{2})$ 1 $(l_1\ l_2\ \delta_l) \leftarrow$ Closest Pair (L_x, L_y)

$T(\frac{n}{2})$ 2 $(r_1\ r_2\ \delta_r) \leftarrow$ Closest Pair (R_x, R_y)

3 $\delta = \min(\delta_l, \delta_r)$

$O(n)$ 4 Compute the 2δ band B_y (Sort B by y coordinates)

~~By~~



Each of the 8 regions he can have only 1 point each.

$8 \times O(n)$

~~T(n)~~ 5 "Inspect" in B_y for points which are at a distance smaller than δ.

6 Return (p, q, δ')

$$T(n) \leq 2T\left(\frac{n}{2}\right) + O(n)$$

maximind $T(n) = n\log n$ $O(n \log n)$

* $\frac{2}{3}$ partitions

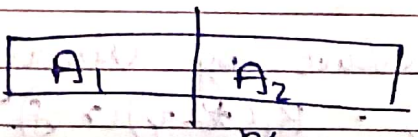
* Multiplication of Polynomials

$A = (a_0, a_1, a_2 \cdots a_n)$
$B = (b_0, b_1, b_2 \cdots b_m)$

$A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n \quad a_n \neq 0$
$B(x) = b_0 + b_1 x + b_2 x^2 + \cdots b_m x^m \quad b_m \neq 0$

$C(x) = A(x) \cdot B(x)$
$C = (a_0 b_0, a_0 b_1 + b_0 a_1, a_0 b_2 + a_1 b_1 + a_2 b_0 + \cdots$

$\cdots, a_n b_m)$

$A(x)$ 

$A(x) = A_1(x) + x^{n/2} A_2(x)$
$B(x) = B_1(x) + x^{n/2} B_2(x)$

$C(x) = A_1(x) B_1(x) + x^{n/2}(A_1(x) B_2(x) + A_2(x) B(x)) + x^n A_2(x) B_2(x)$

$(A_1(x) + A_2(x))(B_1(x) + B_2(x))$

3 calculations

* $O(n^{\log_2 3})$ like in case of integer multiplication

?? Prefix Sum Method

$a_0 \quad b_0 \rightarrow a_0 b_0$

$a_1 \quad b_1 \quad a_0 + a_1 \quad b_0 + b_1 \rightarrow a_0 b_0 + a_1 b_1 + (a_1 b_0 + a_0 b_1)$

$a_0 + a_1 + a_2 \quad b_0 + b_1 + b_2$

Fast Fourier Transform.

Lecture 9

29/1/2024

$$c_i = \sum_{j=0}^{i} a_j b_{i-j}$$

↓

Coefficient of x^i in $C(x) = A(x)B(x)$

⇒ Evaluating ~~in~~ the polynomial at one point

$$p(x) = \underbrace{p_0 + p_1 x + p_2 x^2}_{1+3=4 \text{ mult.}} + \cdots + p_n x^n$$

$$p_0 + (p_1 + p_2 x)x$$

↳ 2 multi.

Evaluating / storing the roots of the polyn with constant

↓

Obtain the product by directly multiplying $K(x-\alpha)(x-\beta) \cdot K_1(x-\gamma)(x-\delta)$.

* Knowing the product at ⓝ most n+1 points.

E.g. Degree 2

$$a_0 + a_1 x + a_2 x^2$$

$$p(0) = 1 \quad p(1) = 2 \quad p(18) = 3$$

Vander monde matrix

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

* Evaluating the polynomial at special points → Roots of unity.

* E.g. $a_0 + a_1 x + a_2 x^2 + a$

(a_0, a_1, a_2) Evaluating at 4 different roots of unity

$\pm 1 \quad \pm i$

$$\begin{bmatrix} a_0+a_1+a_2 \\ a_0-a_1+a_2 \\ a_0+a_1 i-a_2 \\ a_0-a_1 i-a_2 \end{bmatrix}$$

* $T(n)$: Cost of evaluating a poly: of degree : n at 'n' diff. values

Assume ~~of~~ for analysis n is power of 2

$$P(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_n x^n$$

$$= \left(p_0 + p_2 x^2 + p_4 x^4 + \cdots p_k x^{2k}\right) +$$

$$\left(p_1 x + p_3 x^3 + \cdots p\right)$$

$$= P_{even} + x\,(P_{odd})$$

$$P_{even} = p_0 + p_2 x + p_4 x^2 + p_6 x^3 + \cdots$$

$$P(x) = P_{even}(x^2) + x\,P_{odd}(x^2)$$

P at 16 roots of unity

↓

evaluate P_{even} at 8 roots of unity

P_{odd} at 8 roots of unity

$+ O(16)$ operations

$T(n) \leq 2.T(n/2) + O(n)$

$\Rightarrow T(n) = O(n \log n)$

Lecture 10

30/1/2024

* We evaluated our polynomial on 2^K roots of unity where 2^K is just less than n (degree of polynomial).

Eg. n = 63 → 64 roots of unity.
16 → 16 roots of unity

After calculating the values we can obtain the polynomial by taking inverse of coefficient matrix.

* $\pm 1, \pm i$ (last Example)

$$\begin{bmatrix} a_0+a_1+a_2 \\ a_0-a_1+a_2 \\ a_0+a_1 i-a_2 \\ a_0-a_1 i-a_2 \end{bmatrix}$$

no. of roots of unity where evaluation is done

$2K \leq n$ (n is a power of 2)

$A \cdot B = C$

$(a_0\ a_1 \cdots a_K)(b_0\ b_1 \cdots b_K) =$

Values of c at $1, \omega, \omega^2, \ldots \omega^{n-1}$

$$[c^1, c^\omega, c^{\omega^2}, \ldots c^{\omega^{n-1}}]$$

$c(1)$ $c(\omega)$ ...

$D(x) = c^1 + c^\omega x + c^{\omega^2} x^2 + \cdots + c^{\omega^{n-1}} x^{n-1}$

$$D(\omega^i) = \sum_{i=0}^{n-1} c^{\omega^i} x^i \Big|_{x=\omega^i}$$

$C^{\omega^k} = C(\omega^k)$

$= c_0 + c_1\omega^k + c_2\omega^{2k} + \ldots\ldots$

$$D(\omega^j) = \sum_{i=0}^{n-1} \sum_{t=0}^{n-1} c_t \omega^{it} \omega^{ji}$$

$$= \sum_{t=0}^{n-1} \sum_{i=0}^{n-1} c_t \omega^{i(j+t)}$$

$$= \sum_{t=0}^{n-1} c_t \sum_{i=0}^{n-1} \left[\omega^{(j+t)}\right]^i$$

all other will vanish except c_{n-j}

$t = n-j$

$= c_{n-j} \times n$ $\quad$ j varies from 1 to n

$$c_{n-j} = \frac{D(\omega^j)}{n}$$

$c_0 \cdot n \times$ Check() $\quad$ Boolean function returns 0 or 1

$c_1 \cdot n \times$ Check()

$\omega^{t+j} = \alpha^r$

Lecture 11

31/1/2024

* Matrix Multiplication

Input size $O(n^2)$

Time Complexity → $O(n^3)$ Normal Multiplication

$$\begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} = \begin{bmatrix} A_1B_1 + A_2B_3 & \\ & \end{bmatrix}$$

$A_1B_1 + A_2B_3$ ↓ addition of $\frac{n}{2} \times \frac{n}{2}$ matrices $\Rightarrow O(n^2)$

$$T(n) = 8T(n/2) + O(n^2)$$

* Strassen's matrix multiplication

Similarly like a previous cases

$(A_1+A_2)(B_1+B_3) =$

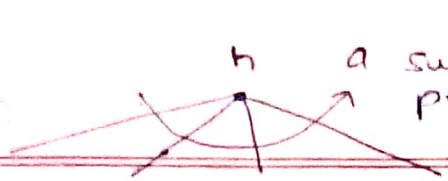
7 multiplications and C_i's are extracted from manipulation

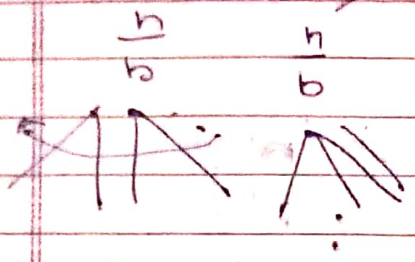
$$\Rightarrow T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

$$T(n) = O(n^{\log_2 7})$$

* Master's Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + g(n)$$





$a \cdot g\left(\frac{n}{b}\right)$

$a^2 \cdot g\left(\frac{n}{b^2}\right)$

$a^k g\left(\frac{n}{b^k}\right)$

$\left(\frac{n}{b^k}\right)$ → Final stage

$= a^k g(1) = a^k$

$b^k = n \Rightarrow k = \log_b n$

Eg $g(n) = n^2$

n^2

$a\, g\left(\frac{n}{b}\right)^2$

$a^2 \left(\frac{n}{b^2}\right)^2$

$$n^2\left[1 + \frac{a}{b^2} + \left(\frac{a}{b^2}\right)^2 + \left(\frac{a}{b^2}\right)^3 + \cdots\right]$$

If $\frac{a}{b^2} < 1$ ↓ Constant

$\Rightarrow cn^2$

$\frac{a}{b^2} = 1 \Rightarrow n^2[1 + 1 + 1 + \cdots]$

↓ No. of layers

$= n^2 \log_b n$ " $\log_b n$

$\frac{a}{b^2} > 1$

$n^2 \left(1 + \frac{a}{b^2} \cdots a^K \left(\frac{n}{b^K}\right)^2\right) \cdots$

Dominating factor.

$\Rightarrow n^2 \frac{a^{\log_n b}}{\left(b^{\log_n b}\right)^2}$ =

$\Rightarrow n^2 \left(\frac{a}{b^2}\right)^{\log_b n}$

$= (n^2) \frac{a^{\log_b n}}{\left(n^{\log_b b^2}\right)} = n^{\log_b a}$

Lecture 12 • 5/2/2024

Some Problem Discussion Missed

* Greedy Algorithm

Stable Marriage Problem
↳ Objective:- Matching man and woman to each other
n Men and n Women (Each person has a set of preferences)

Extension of the problem:- Jobs and Applicants
If Jobs ≠ Applicants then create dummy jobs which does not pay so that reduction of the extension to the normal problem

Preference in form of permutation

m_1 w_1 w_{18} w_3 ... w_n w_{n-1}
m_2

m_n

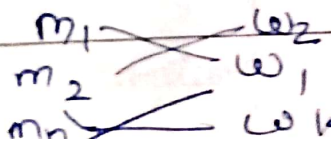
w_1 m_{18} m_{21} m_4 ... m_2

w_n

Objective:- Find a matching without instabilities

Def^n:- (m, w) is an unstable pair if the following holds:- ↓ in a matching (M)

⇒ m prefers w to
w prefers m
over their current



* Find matchings (preference orders) so that ∄ a unstable pairs?
* In certain cases of pref. orders there will exist ∞ O unstable pairs.

Eg.

$m_i - w_i \quad \forall \; i \;\; s.t \;\; 1 \le i < n.$

Lecture 13

6/2/2024

* m_1 m_2 ... m_n — w_1 w_2 ... w_n

* Preference Order

m_1	w_1 w_2 w_3		w_1	m_2 m_1 m_3	
m_2	w_2 w_1 w_3		w_2	m_1 m_3 m_2	
m_3	w_3 w_2 w_1		w_3	m_2 m_3 m_1	

Is (w_2, m_1) unstable ⇒ Give other parameter matching ←

M

$m_1 - w_1$ Given M, $[w_2, m_1]$ is unstable

$m_2 - w_2$ ~~the~~ bcoz m_1 has w_1

$m_3 - w_3$ his 1st preference.

* Pair unstable if both of them prefer each other over their current pref.

* There is an algorithm which proves that given any preference order there exists a matching which has no unstable pairs.

w_1 — $\nabla_1(n)$

w_2 — $\nabla_2(n)$

: w_i — $\nabla_i(n) = (m_3\ m_2 \cdots m_n)$

w_n — $\nabla_n(n)$

Women initiate proposal and men accept better proposal.

The algo indeed terminates and it terminates with a matching.

This matching has 0 unstable pair.

Let (m, w) unstable pair

w has not prop $\Rightarrow$ w had a better choice where she was accepted

w " " $\Rightarrow$ m had a better choice at the time of proposal

Eg:

m_i	w_2 $\quad$ w_1
m_2	w_3 $\quad$ w_2
m_i	w_{i+1} $\quad$:
m_n	w_1 $\quad$ w_n

w_1	m_1	m_n
w_2	m_2	m_1
w_{i+1}	m_3	m_2
w_n	m_4	m_{i+1}

$w_i - m_i$ No unstabilities

$w_{i+1} - m_i$ No unstabilities

* Can there be 2 matchings at the end of the algorithm?

↳ There will be a unique matching which will have 0 unstability.

Lecture 14 7/2/2024

* Algorithm of Stable Matching:

Stable Matching (P)

Repeat until convergence

1. Arbitrarily choose a woman w who is not engaged + she proposes to 1st available man say m

2. m accepts if unengaged or is partnered with a less preferable person

return the matching that was created

Algo terminates bcoz women can make at most n proposals

All people are matched bcoz a matched man remains matched

* Only way a man remains unmatched is no proposals which is not possible

O unstability ⇒ Given on previous page

* Unique Matching

In the set of all n! matching, collect all stable matchings ~~find all f~~
From all stable matchings find

Also

w_1	Best(w_1)	m_1	Worst(w_1)
w_i	Best(w_i)	m_i	
w_n	Best(w_n)	m_n	Worst(w_n)

w – Best of (w) = m

* Claim:- If m' rejects w then m' is infeasible; for w

No stable matching exists

⇒ m' prefers current partner w' over w

We wish to show that there are no stable matching with (m', w)

$w' > w$ m' – w Let it be for contradiction

$m^1 \to m^2$ $m^2 - w'$

because m^2 has rejected w' earlier

12/2/2024

Lecture 15

5 ☐ 2x or x/2 → E(Money) = 1.25x

Stable Marriage Problem Continued

* Proof of $Best(p_1) \neq Best(p_2)$

Stable matching m_1 s.t. m_2

w_1 | x | w_2 | y |

x cannot be equal to y.

$Best(p) = \{ m(p) \mid m \in \mathcal{M} \}$ (max according to preference given by p.)

$\mathcal{M} = \{ m \mid m \text{ is a stable matching} \}$ → Feasible set for p

$p_1 \neq p_2 \Rightarrow Best(p_1) \neq Best(p_2)$

claim.

* If a man (m) rejects a woman (w) in GS algo then m is infeasible for w.

During GS execution w was rejected by m

w − m

Reason:- m got proposed from w'

m

w − m

w' − m'

w' prefers m' over m for m to be stable. ⇒ w' must have proposed m' earlier than m. Since w' proposed m

maximind

Scanned with OKEN Scanner

this shows that m' must have rejected w' in one of the earlier rounds.

13/2/2024

Prove that G S Algo matches every man to m to worst(m). = w.

Worst(A) is the worst preferred partner for A among all his/her partner in stable matchings.

G S returned	Other stable Matchings
m — w	m : w'
	m' — w

m was best for w so w prefers m > m' but what makes it a stable matching is w' > w ⇒ m got w which is worst possible match.

Lecture 16 13/2/2024

ASCII → 8 bit

Character set = {0, 1}

$2^8 = 256$ different characters

For having a coding so that I can directly convert each bit into value without worrying about prefix. Prefix freeness is necessary.

Prob. Code Word (Binary and Prefix free)

a → p_1 c_1

z → p_{26} c_{26}

$$\text{Cost} = \sum_k p_k |c_k| := \sum_k f_k |c_k|$$

$|c_k|$ → Length

Obj. – minimize Cost

Notes missed

Lecture 14

14/2/2024

For encoding symbols in binary.

no. of symbols = height of binary tree

$\binom{2^K}{K}$ possible encodings

For each symbol find a node in binary complete tree so that no 2 share ancestor-descendent relationship. → Also optimize cost.

↳ Prefix free:

* Combining 2 symbols into one

$a_1 \quad a_{K-1} \; a_K$

$e_B \cdot 0 \quad e_B \cdot 1$

$a_1 \quad \cdots \quad a_{K-2} \; (B) \rightarrow a_{K-1} \; a_K$

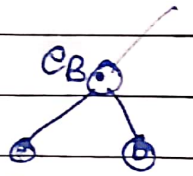
$e_1 \quad e_{K-2} \; e_B$

The 2 symbols are least freq. ones

Arguing for Prefix free

$a_1 \cdots a_{K-1}$ → Prefix free

P-free



Adding 0 and 1 makes node down so no conflict.

Not necessary that least 2 freq. symbols will be siblings.

$f_i \times h_i + f_j h_j$

$f_i \times h_j + f_j h_i$

In Optimimal Codings

Symbol	Freq	Height (Depth)
	smallest f_i	largest h_i
	f_i	h_j
	largest	smallest

↓ Non increasing

If $h_i < h_j$

After Swapping Increase $f_i \Delta h$

Decrease $f_j \Delta h$

Net Change $(f_i - f_j)\Delta h < 0$

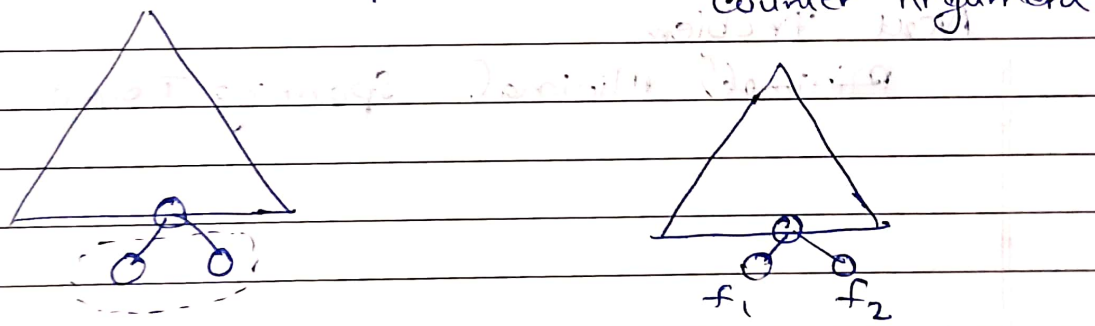
Thus initial encoding was not optimal

* Proof of Optimal Trees

① Ensure that B will be at the bottom

Returned by Recursion ⇒ Optimal

Counter Argument



B → Optimal tree with node B

$B + f_1 + f_2$ is the cost of a tree for $(a_1 \cdots a_k)$

Optimum for $a_1 \cdots a_k$ costs C

$C - f_1 - f_2$ is a tree for $(a_1 \cdots a_{k-2}, B)$

Its costs D is $C - f_1 - f_2 \geq B$

$C \geq B + f_1 + f_2$

We have already created a tree which costs $\beta + f_1 + f_2$ ⇒ Thus this one is the optimal which we used as counter-argument.

* Time Complexity

Maintain a min-heap for extracting 2 ~~no~~ least frequent ⇒ sum them up and put back.

2n extract
n insertions → 1 node tree

Then create the encoding.

Next Problem

~~Minimal~~ Minimal Spanning Tree

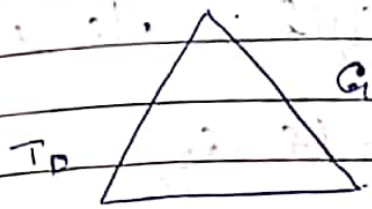
Lecture 18

17/2/2024

Huffman tree on K nodes

$(f_n, f_{n-1}, \cdots f_3, f_2+f_1)$ — K

i

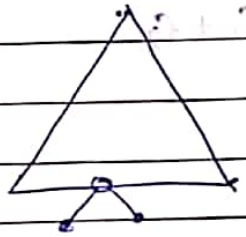


Guaranteed to be optimal because it is returned by recursion

This tree is best for K freq. above

Proof of correctness for last 2 nodes & Σ K

ii

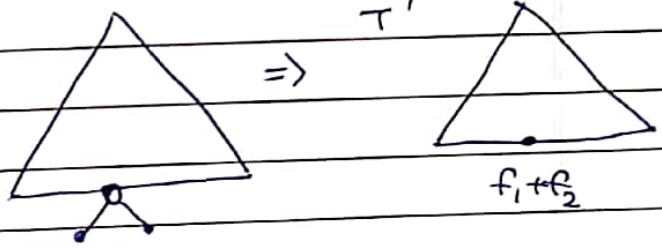


K+1 node tree returned by the algo

Claim: For K+1 freqs $(f_n, f_{n-1}, \cdots f_3, f_2, f_1)$ the tree T_2 is optimal

Consider the tree that is optimal for $(f_n, f_{n-1}, \cdots f_1)$

T_{opt}



T' is a valid prefix free encoding for $(f_n, f_{n-1}, \cdots f_3, f_2+f_1)$

Cost (T') $\geq$ Cost (T_0)

maximind

$$Cost(OPT) = \sum_{i} f_i l_i - \sum_{i+2}^{i+1} f_i l_i + f_1 l_1 + f_2 l_2 + (f_1 + f_2)(l+1)$$

$$Cost(T_{opt}) - f_1 - f_2 \geq Cost(T_0)$$

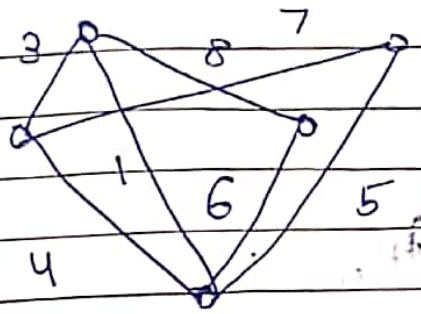
$$Cost(T') = OPT - (f_1 + f_2)(l+1) + (f_1 + f_2) l$$

$$= OPT - (f_1 + f_2)$$

$$Cost(T_{opt}) \geq Cost(T_0) + f_1 + f_2$$

$$OPT_{k+1} \geq OPT_k + f_1 + f_2$$

* Graph - Weighted



Q Number of Spanning Trees of a Complete Graph?

Heuristics for MST

i For any cycle heaviest edge will be ~~odd~~ absent

ii Split graph into 2 parts and least connecting edge must be present in every spanning tree.

20/2/2024
Lecture 19

Minimal Spanning Tree Algorithms

Assumption :- (i) All edges weights are distinct and +ve.

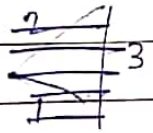
(ii) The input graph G_1 is connected

MST : A least wt tree of G_1.

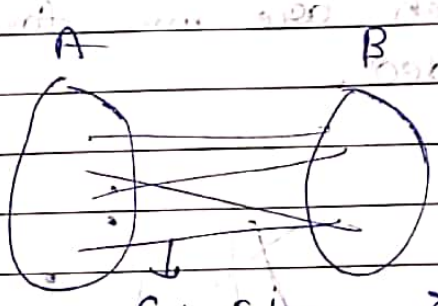
⇒ Arthur Merlin Problem
: Similar to matching Problem

Adjacency Matrix

Permanence → Remove (-1) power just add all values.



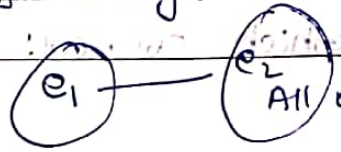
Is the MST unique ?



Cut Edges → weighted

The least cut edge is invariably present in all the possible minimal spanning tree.

The least weighted edge also must be present



maxmind

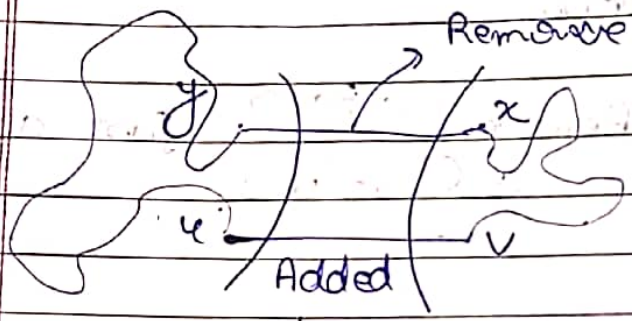
Cycle Property

The max weighted edge of any cycle is guaranteed to be not present in any MST.

* Proof (Cut Proposition)

Least weighted edge $\therefore (u,v) \in (A,B)$ is the cut

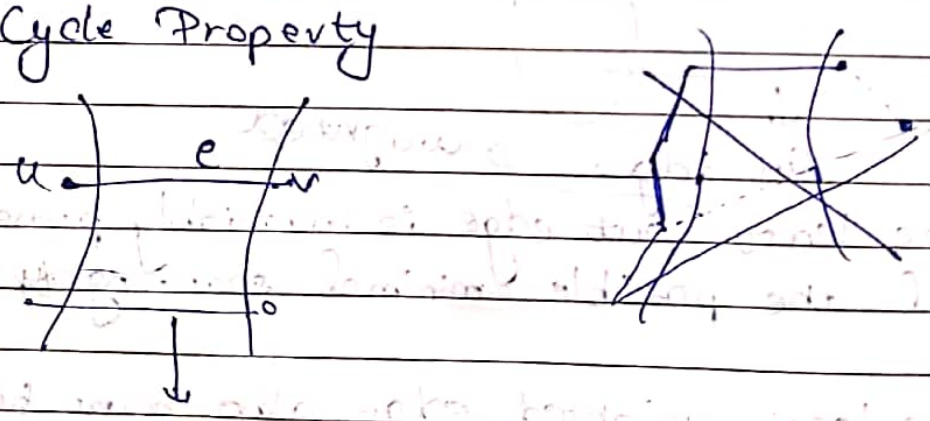
MST : t doesn't have e



For moving using x y we u-v now.

⇒ All edges which are min for a vertex gets added.

* Cycle Property



Add this edge which connects this 2 groups

21/2/2024

* Any edge which is min weighted cut edge must should be present. → Calc.
  ↳ Distinct values not necessary

* Number of cuts in a graph $= 2^{n-1} - 1$

$$\frac{2^n - 2}{2} = 1$$

* In a Graph any.

Probability that any edge will bet cut edge $= \frac{1}{2}$

Expect. No. of cut edges $= \frac{n}{2}$

Any min weit. cut edge will be present in any one of MST.

* Cycle Property

If MST which doesn't contain max wted edge.

** Kruskal's Algorithm.

Sort edges in increasing order of weight.

mlogm m = # of edges

Add an edge to T as long as no cycles are created.

* Kruskal's Algo

sort the edges
add edges least ⇒ next ⇒ as long as cycle is not created.

* Prim's Algo

For each vertex add its nearest neighbour do it for all the vertices. You get the MST.

* Borauka's Algo

1st Phase → For every vertex add its nearest neighbour to form connected components

2nd Phase. Form supernodes then treat them as cut parts to find min cut edge.

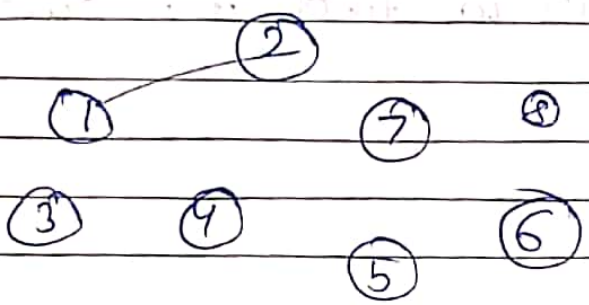
* Lot of MCQs

Date / /
Page

Argument for ST - Any edge which is thrown away doesn't affect connectivity.

Checking for ~~connect~~ cycles

* DSU (Disjoint Set Union)



{1, 2} {3}
{4, 7} {5} {6} {8}

Time Comp: ⇒ O(m). Union and Find
O(m log n)

* Prim's Algo

Start vertex v
Add v to a set S.
Compute nearest neighbour of S. Add that edge to ⊗

Add u to S

O(m). log m

* Reverse Delete
↳ Take the whole graph → Start from heaviest wt. ⇒ Remove them if it is a part of a cycle.

maximind